Univerzita Karlova

Přírodovědecká fakulta

Studijní program: Geoinformatika, kartografie a dálkový průzkum Země

Jan Bartůšek, Petr Havel

Aplikace počítačové kartografie

Geometrické vyhledávání bodu

1. Zadání

Vstup: Souvislá polygonová mapa n polygonů $\{P_1, ..., P_n\}$, *analyzovaný bod* q.

Výstup: P_i, $q \in P_i$.

Nad polygonovou mapou implementujete Ray Crossing Algorithm pro geometrické vyhledání incidujícího polygonu obsahujícího zadaný bod q.

Nalezený polygon graficky zvýrazněte vhodným způsobem (např. vyplněním, šrafováním, blikáním). Grafické rozhraní vytvořte s využitím frameworku QT.

Pro generování nekonvexních polygonů můžete navrhnout vlastní algoritmus či použít existující geografická data (např. mapa evropských států).

Polygony budou načítány z textového souboru ve Vámi zvoleném formátu. Pro datovou reprezentaci jednotlivých polygonů použijte špagetový model.

Hodnocení:

Krok	**Hodnocení**
Detekce polohy bodu rozlišující stavy uvnitř, vně polygonu.	10b
Analýza polohy bodu (uvnitř/vně) metodou Winding Number Algorithm.	*+10b*
Ošetření singulárního případu u Winding Number Algorithm: bod leží na hraně polygonu.	*+5b*
Ošetření singulárního případu u Ray Crossing Algorithm: bod leží na hraně polygonu.	*+5b*
Ošetření singulárního případu u obou algoritmů: bod je totožný s vrcholem jednoho či více polygonů.	*+2b*
Zvýraznění všech polygonů pro oba výše uvedené singulární případy.	*+3b*
Rychlé vyhledání potenciálních polygonů (bod uvnitř min-max boxu).	*+10b*
Řešení pro polygony s dírami.	*+10b*
*Načtení vstupních dat ze *.shp.*	*+10b*
Max celkem:	**65b**

2. Bonusové úlohy a bodové hodnocení

V rámci řešení se podařilo úspěšně implementovat následující bonusové úlohy:

Detekce polohy bodu rozlišující stavy uvnitř, vně polygonu (10b)

Analýza polohy bodu (uvnitř/vně) metodou Winding Number Algorithm (+10b)

Ošetření singulárního případu u Winding Number Algorithm: bod leží na hraně polygonu (+5b)

Ošetření singulárního případu u Ray Crossing Algorithm: bod leží na hraně polygonu (+5b)

Ošetření singulárního případu u obou algoritmů: bod je totožný s vrcholem (+2b)

Zvýraznění všech polygonů pro oba výše uvedené singulární případy (+3b)

Rychlé vyhledání potenciálních polygonů (bod uvnitř min-max boxu) (+10b)

Řešení pro polygony s dírami (+10b)

Načtení vstupních dat ze shp (+10b)

Celkové možné hodnocení: 65 bodů

3. Teoretická část

Úloha spadá do kategorie Point Location Problem, který se často řeší v prostředí GIS pro rychlé určení polohy bodu vůči jiným objektům. V aplikaci jsme implementovali dva odlišné přístupy pro takzvanou lokální proceduru, tedy opakované určení polohy bodu q vzhledem k mnohoúhelníku.

Ray Crossing Algorithm

Tato metoda je založena na principu, kdy je testovaným bodem q vedena polopřímka r, takzvaný paprsek. U této polopřímky se zjišťuje počet jejích průsečíků k s oblastí P. Pokud je tento počet průsečíků lichý ($k\%2 = 1$), bod q leží uvnitř polygonu. Pokud je počet průsečíků sudý nebo nulový, bod q leží vně polygonu. Abychom zjednodušili výsledné vztahy a ošetřili singularity, implementovali jsme variantu s redukcí souřadnic k bodu q. V této variantě posouváme počátek lokálního souřadnicového systému do bodu q.

Souřadnice vrcholů p_i redukujeme odečtením souřadnic bodu q: $x_i' = x_i - x_q \; a \; y_i' = y_i - y_q$.

Rovnice paprsku je v tomto systému definována prostě jako $y' = 0$.

Test existence průsečíku paprsku s hranou mezi vrcholy se tím pádem zjednoduší na test, zda hrana protíná tuto osu: $(y_i' > 0) \wedge \left(y_{\{i-1\}}' \leq 0\right) \vee \left(y_{\{i-1\}}' > 0\right) \wedge (y_i' \leq 0)$.

Samotná x-ová souřadnice průsečíku hrany s paprskem x_m' se vypočítá vztahem:

$$x_m' = \frac{\left\{x_i' y_{\{i-1\}}' - x_{\{i-1\}y_i'}'\right\}}{\left\{y_i' - y_{\{i-1\}}'\right\}}$$

Průsečík je započítán (inkrementace k = k + 1) pouze v případě, že leží v pravé polorovině, tedy pokud $x_m' > 0$.

Winding Number Algorithm

Tato metoda vyhodnocuje součet všech rotací ω_i , které musí průvodič z bodu q opsat nad všemi vrcholy p_i z polygonu P.

Úhly mají definovanou orientaci. Úhel ω má kladné znaménko při směru CCW (proti směru hodinových ručiček) a záporné při směru CW.

Celkový součet rotací udává takzvaný Winding Number, počítaný jako:

$$\Omega(q, P) = \frac{1}{2\pi} \sum_{\{i=1\}}^{\{n\}} \omega\left(p_i, q, p_{\{i+1\}}\right)$$

Pokud analyzovaný bod q leží uvnitř mnohoúhelníku, pozorovatel se musí otočit o plný úhel 2π a hodnota $\Omega = 1$.

Pokud bod q leží vně mnohoúhelníku, tento úhel je menší než 2π a $\Omega = 0$.

Vzhledem k nepřesnostem při výpočtech v plovoucí řádové čárce se pro test, zda bod leží uvnitř, používá test odchylky od celého úhlu $2\pi \; s \; tolerancí \; \epsilon$, tedy $|\Omega - 2\pi| < \epsilon$.

Min-Max Box a Komplexní polygony

Před zahájením výpočtu lze aplikovat takzvaný Bounding Box (obálkovou metodu) k rychlé eliminaci polygonů, v jejichž blízkosti se bod q vůbec nenachází. U non-simple oblastí s vnořenými polygony (dírami) se Ray Crossing aplikuje nativně na všechny hrany (řídí se pravidlem Even-Odd), zatímco pro

Winding Number platí, že bod je uvnitř polygonu pouze tehdy, pokud leží v primární hranici a zároveň neleží uvnitř žádné z definovaných děr.

4) Implementace a rozbor kódu v Pythonu

Aplikace je vyvinuta v jazyce Python s využitím frameworku PyQt6. Architektura je striktně rozdělena na tři moduly: MainForm.py (uživatelské rozhraní a aplikační logika), draw.py (vykreslovací plátno a interakce myší) a algorithms.py (geometrická a matematická logika).

4.1 Datový model a zobrazení děr (Třída Draw)

Základním požadavkem pro bonusové body bylo zvládnutí tzv. *komplexních polygonů* (polygonů s dírami). V modelu jsme proto přešli z objektu QPolygonF na vícerozměrné pole list[list[QPolygonF]].

Každý prvek mapy (komplexní polygon) je reprezentován jako seznam list.

Nultý index tohoto seznamu (complex_pol[0]) vždy představuje vnější hranici (outer boundary).

Indexy 1 až n (complex_pol[1:]) reprezentují případné vnitřní díry (holes).

K vizualizaci vyříznutých děr na plátně využíváme nativní třídu QPainterPath z frameworku Qt, které nastavíme pravidlo pro vykreslování křížících se polygonů:

```
# QPainterPath for rendering holes with OddEvenFill rule
path = QPainterPath()
path.setFillRule(Qt.FillRule.OddEvenFill)

# Add outer boundary and all holes to the path
for ring in complex_pol:
    if not ring.isEmpty():
        path.addPolygon(ring)

qp.drawPath(path)
```

4.2 Min-Max Box (Rychlé předvyhledávání)

Abychom ušetřili výpočetní čas při velkých datasetech, implementovali jsme ve třídě Algorithms funkci isInBoundingBox. Funkce získá vrcholy pouze z vnější hranice (complex_pol[0]) a najde jejich extrémy.

```
def isInBoundingBox(self, q: QPointF, complex_pol: list[QPolygonF]):
    # Min-Max Box check the outer boundary
    if not complex_pol or complex_pol[0].isEmpty():
        return False

    outer_ring = complex_pol[0]

    x_coords = [p.x() for p in outer_ring]
    y_coords = [p.y() for p in outer_ring]

    min_x, max_x = min(x_coords), max(x_coords)
    min_y, max_y = min(y_coords), max(y_coords)

    if q.x() < min_x or q.x() > max_x or q.y() < min_y or q.y() > max_y:
        return False
    return True
```

4.3 Ošetření singularit a uživatelského vstupu (Tolerance)

Důležitým aspektem návrhu bylo ošetření singularit. Zavedli jsme proměnnou tolerance = 5.0 (buffer v pixelech) k zachycení nedokonalých kliknutí myší.

```
            if math.hypot(xi, yi) <= tolerance:
                return -2

            dx = xi1 - xi
            dy = yi1 - yi
            edge_length = math.hypot(dx, dy)

            if edge_length > 0:
                distance_to_line = abs(xi * yi1 - xi1 * yi) / edge_length
                if distance_to_line <= tolerance:
                    dot_product = -(xi * dx + yi * dy)
                    if 0 <= dot_product <= edge_length**2:
                        return -1
```

4.4 Ray Crossing Algorithm

Pokud bod nepatří mezi singularity, Ray Crossing pokračuje hledáním průsečíku hrany s paprskem. Zde je naše implementace vycházející z rovnice y' = 0 aplikovaná na redukované souřadnice:

```
                if (yi1 > 0 and yi <= 0) or (yi > 0 and yi1 <= 0):
                    xm = (xi1 * yi - xi * yi1) / (yi1 - yi)
                    if xm > 0:
                        k += 1

        if k % 2 == 1:
            return 1
        return 0
```

4.5 Winding Number Algorithm

Abychom podpořili komplexní polygony, vytvořili jsme pomocnou metodu _getWindingNumberForRing, kterou voláme nejprve pro vnější hranici a následně pro díry:

```
def getPointPolygonPositionWN(self, q: QPointF, complex_pol: list[QPolygonF]):
    # Analyze using Winding Number
    if not complex_pol or complex_pol[0].isEmpty():
        return 0

    # 1. Check outer boundary
    outer_status = self._getWindingNumberForRing(q, complex_pol[0])

    if outer_status != 1:
        return outer_status

    # 2. Check all holes
    for hole in complex_pol[1:]:
        if hole.isEmpty():
            continue

        hole_status = self._getWindingNumberForRing(q, hole)

        if hole_status == -1 or hole_status == -2:
            return hole_status # On the edge/vertex of a hole

        if hole_status == 1:
            return 0 # Inside a hole

    return 1
```

Samotný výpočet v metodě _getWindingNumberForRing je plně vyřešen funkcí math.atan2 pro korektní úhlovou orientaci:

```
        cross_product = xi * yi1 - xi1 * yi
        dot_product = xi * xi1 + yi * yi1

        angle = math.atan2(cross_product, dot_product)
        omega += angle

    if abs(abs(omega) - 2 * math.pi) < 1e-5:
        return 1
    return 0
```

4.6 Import a transformace souborů Shapefile (.shp)

K načítání geografických dat slouží knihovna pyshp. Parser ve třídě MainForm.py dynamicky načte polygony, oddělí jejich *parts* (díry) a matematicky je transformuje, včetně scale a převrácení osy Y z geografického formátu pro vykreslování v obrazovkových souřadnicích.

```
shapes = sf.shapes()

loaded_polygons = []

# Bounding box of the entire shapefile for scaling
min_x, min_y, max_x, max_y = sf.bbox
width = max_x - min_x
height = max_y - min_y

# Prevent division by zero if shapefile is a single point
if width == 0: width = 1
if height == 0: height = 1

# Scale coordinates to fit roughly into an 800x800 pixel window
scale_x = 800 / width
scale_y = 800 / height
# Keep aspect ratio by using the smaller scale factor, and leave a 5% margin
scale = min(scale_x, scale_y) * 0.95

for shape in shapes:
    # ShapeType 5 means Polygon
    if shape.shapeType != 5:
        continue

    points = shape.points
    parts = list(shape.parts)
    # Add the total number of points as the final boundary index
    parts.append(len(points))

    complex_pol = []
    # Iterate over rings (0th is usually outer, subsequent are holes)
    for i in range(len(parts) - 1):
        start_idx = parts[i]
        end_idx = parts[i+1]
        ring_points = points[start_idx:end_idx]

        ring = QtGui.QPolygonF()
        for px, py in ring_points:
            # Translate point to origin, scale it, and offset to center
            screen_x = (px - min_x) * scale + 20
            # Y axis is flipped in screen coordinates compared to geography
            screen_y = 850 - ((py - min_y) * scale + 20)
            ring.append(QtCore.QPointF(screen_x, screen_y))

        complex_pol.append(ring)

    loaded_polygons.append(complex_pol)
```

4. Závěr

Úloha byla úspěšně realizována se ziskem plného počtu bodů na obou frontách – matematické (kompletní implementace algoritmů a jejich ošetření proti singularitám) i softwarové (plynulé PyQt rozhraní, transformace souřadnic z geografie na plátno a robustní paměťový model).

Optimalizace datových struktur a implementace obálkové metody (Min-Max Box) zaručují, že řešení je okamžité i u map obsahujících stovky tisíc bodů. Během testování byl s úspěchem načten a plnohodnotně analyzován referenční dataset administrativního členění států světa z projektu *Natural Earth*, u nějž byly oběma algoritmy bezchybně analyzovány složité hranice i s četnými mezinárodními enklávami a vnitřními ostrovy (dírami). Změna logiky z QPolygonF na vícerozměrná pole (pro podporu děr) činí aplikaci plně způsobilou pro profesionální topologické úlohy v rovině. Aplikace prokazatelně zvládá ošetření veškerých hraničních případů i import pokročilých .shp formátů.

5. Seznam literatury

1. BAYER, Tomáš. *Point Location Problem. Konvexní a nekonvexní oblasti. Ray Algorithm. Winding Number Algorithm* (Přednáška adk3_new.pdf, Katedra aplikované geoinformatiky a kartografie, Přírodovědecká fakulta Univerzity Karlovy).